Guía Definitiva de EtherNet/IP y el Protocolo Industrial Común (CIP) (Versión Definitiva)

Este documento es una guía estructurada y detallada que resume los fundamentos y los conceptos avanzados de EtherNet/IP y su protocolo subyacente, CIP. Está diseñado para servir como un pilar de conocimiento para la formación, referencia y como base de conocimiento para futuros análisis.

Parte 1: El Corazón - El Protocolo Industrial Común (CIP)

El error más común es confundir CIP y EtherNet/IP. La distinción es la base de todo el ecosistema y es crucial para entender cómo interactúan los dispositivos.

- **CIP (Common Industrial Protocol):** Es el **"idioma"** 💬. Es un protocolo de capa de aplicación (Capa 7 del modelo OSI), orientado a objetos, que define *qué* decimos, *cómo* organizamos los datos y *qué* podemos hacer con ellos. Es una especificación gestionada por la organización **ODVA (Open DeviceNet Vendors Association)**. El objetivo de CIP es ser "Común" para romper los silos de los protocolos propietarios (donde un dispositivo de Marca A no podía hablar con uno de Marca B) y permitir la interoperabilidad. CIP, al ser un protocolo de Capa 7, es puramente una especificación de datos y servicios, completamente independiente del hardware de red.
- **EtherNet/IP (Industrial Protocol):** Es el **"método de transporte"** 🚚. Es la implementación específica de CIP *sobre* la red Ethernet estándar (IEEE 80.2) utilizando la suite de protocolos TCP/IP. La "IP" significa "Protocolo Industrial", no "Protocolo de Internet". EtherNet/IP actúa como el "adaptador" que toma el "paquete" de datos CIP de la Capa 7 y lo envuelve en un datagrama UDP (para control en tiempo real) o en un segmento TCP (para configuración), que luego son envueltos en un paquete IP (Capa 3) para ser enrutados.

Capítulo 1.0: La "Familia" CIP (Por qué se llama "Común")

Antes de profundizar en el modelo de objetos, es vital entender *por qué* CIP se llama "Común". CIP fue diseñado como un "idioma" independiente del "transporte". ODVA estandarizó el idioma (CIP) y luego lo adaptó a múltiples redes físicas (los transportes).

Piensa en CIP como el "español". Puedes hablar español por teléfono, por carta, por email o en persona. El idioma es el mismo, pero el método de entrega es diferente. Esto es la "Familia CIP":

- **DeviceNet:** Es CIP hablando sobre un bus **CAN (Controller Area Network)**. Es una red

de bajo coste, robusta, que utiliza un cable de 4 o 5 hilos que puede **suministrar energía** a los dispositivos (como sensores o fotocélulas), simplificando enormemente el cableado. Es más lento, pero ideal para el nivel de "dispositivo".
    - *Implicación:* La naturaleza abstracta de CIP tiene un coste. Enviar un paquete CIP sobre CAN (un protocolo muy estricto y de paquetes pequeños) añade una sobrecarga (overhead) significativa comparado con un protocolo nativo de CAN como CANopen.
- **ControlNet:** Es CIP hablando sobre un bus **coaxial** (o fibra). Fue el estándar de alta velocidad *antes* de Ethernet. Es altamente **determinista** (puedes predecir con exactitud cuándo llegará un paquete) y se usaba para control crítico entre PLCs (Peer-to-Peer) y variadores de alta velocidad. Su determinismo provenía de un esquema de "ranuras de tiempo" (time slots) gestionado por la red.
- **CompoNet:** Es CIP hablando sobre un bus plano, optimizado para una alta densidad de conexiones de E/S simples (sensores y actuadores).
- **EtherNet/IP:** Es CIP hablando sobre **Ethernet** (TCP/IP y UDP/IP). Se ha convertido en el estándar moderno porque ofrece la mayor velocidad (Gigabit y más), flexibilidad y se integra directamente con las redes de oficina (IT).

Esta arquitectura obligó a CIP a ser un modelo abstracto (Orientado a Objetos). Un programador que aprende el **Objeto Ensamblado** 📦 de CIP puede configurarlo sin importar si el transporte final será un paquete UDP en EtherNet/IP o un mensaje CAN en DeviceNet. El software del controlador y los *gateways* (puentes) de hardware se encargan de la "traducción" al transporte específico.

Capítulo 1.1: El Modelo de Objetos (El "Índice")

CIP es un protocolo **Orientado a Objetos (OOP)**. Este modelo es lo que le da su poder y flexibilidad. Cada dispositivo (sensor, variador, PLC) se ve a sí mismo como un "archivador" 🗄️ estandarizado. Esto permite que un PLC de "Marca A" sepa exactamente dónde encontrar los datos en un dispositivo de "Marca B", siempre que ambos sigan las mismas "interfaces" (Perfiles).

La terminología se traduce casi directamente desde la Programación Orientada a Objetos:

Concepto OOP (Java)	Terminología CIP (ODVA)	Descripción
public class VFD { ... }	**Clase (Class)**	La **plantilla** o el diseño abstracto de un componente. Define la estructura de datos (atributos) y el comportamiento (servicios) que todas las instancias de

		esta clase tendrán.
VFD miVFD = new VFD();	**Instancia (Instance)**	El **objeto real** creado a partir de esa plantilla. Es la entidad con la que se interactúa en la red. Un dispositivo puede tener múltiples instancias de una clase (ej. un chasis de E/S con 8 tarjetas idénticas).
public int serialNumber;	**Atributo (Attribute)**	Un **dato** (variable o campo) dentro de una instancia. Es el valor real que se desea leer o escribir (ej. Frecuencia, Corriente, N.º de Serie).
public String getSerial() { ... }	**Servicio (Service)**	Una **acción** (método) que se puede invocar en una clase o instancia. Define el comportamiento (ej. Get_Attribute_Single, Set_Attribute_Single, Start, Stop).

Esta arquitectura OOP proporciona beneficios clave del mundo del software:

- **Encapsulación:** El PLC (cliente) no necesita saber *cómo* un VFD (servidor) mide la corriente. Solo necesita "pedir" el Atributo[Corriente] del Objeto Motor. La lógica interna del VFD (ya sea con un sensor de efecto Hall o una derivación de shunt) está encapsulada.
- **Polimorfismo:** Un PLC puede enviar un servicio Start genérico (definido en un Perfil) a un VFD de Marca A o a uno de Marca B. Ambos ejecutarán su propia versión interna de "arranque", permitiendo al PLC tratarlos de forma idéntica.

Clases (Objetos) Fundamentales

ODVA define "clases" estándar que los dispositivos deben implementar. Son los "paquetes base" del ecosistema CIP:

1. **Objeto Identidad (Identity Object - Clase 0x01)** 🆔
   - Es el "DNI" del dispositivo. Todo dispositivo CIP debe tener una (y solo una) instancia de esto.

- **Atributos clave:** VendorID (Fabricante), DeviceType, ProductCode, SerialNumber, Revision (Versión).
- **Uso Práctico (Electronic Keying):** Su uso más importante es la "validación" o *Keying*. Cuando un PLC se conecta, "entrevista" al dispositivo y compara su VendorID, ProductCode y Revision con los que están configurados en el proyecto. El ingeniero puede configurar el nivel de esta "llave electrónica":
  - **Keying Exacto (Exact Match):** El dispositivo *debe* coincidir con la Revision exacta (ej. v2.001). Si un técnico instala un v2.002, la conexión falla. Esto es crucial en aplicaciones validadas (ej. farmacéutica).
  - **Keying Compatible (Compatible Match):** El dispositivo debe coincidir en VendorID y ProductCode, pero la Revision puede ser mayor o igual (ej. v2.x). Esta es la opción más común y flexible.
  - **Keying Deshabilitado:** El PLC se conectará a cualquier cosa en esa IP. Es peligroso y no se recomienda.
- **Consecuencia:** Un "fallo de keying" es una de las causas más comunes de error al poner en marcha una máquina.

2. **Objeto Ensamblado (Assembly Object - Clase 0x04)** 📦
   - Es el **DTO (Data Transfer Object)** de CIP.
   - Su propósito es agrupar ("ensamblar") múltiples atributos en un solo paquete de datos (byte[]) para una transferencia de red eficiente. Es la base de la mensajería implícita.
   - Técnicamente, un Ensamblado es una "vista" o un conjunto de "punteros". Una instancia de Ensamblado *no almacena datos*. Es un array de punteros a atributos en otros objetos. Por ejemplo:
     - Ensamblado[0] -> Objeto[Motor](0x29).Atributo[5]
     - Ensamblado[1] -> Objeto[Control](0x2A).Atributo[2]
   - **Implicación:** Esto es increíblemente eficiente. Cuando el dispositivo necesita "producir" un paquete UDP, su CPU no "construye" un paquete; simplemente itera a través de su lista de punteros, recoge los valores de la memoria y los envía.
3. **Objeto Gestor de Conexión (Connection Manager - Clase 0x06)** 🤝
   - Es el "gestor de sesiones" y el "portero" (gatekeeper) de los recursos en tiempo real.
   - Este objeto es el responsable de **activar y desactivar** el flujo de datos en tiempo real (Mensajería Implícita).
   - Es el destinatario del servicio Forward_Open, que es la "solicitud de contrato" para iniciar el streaming de datos.
   - **Implicación:** Un dispositivo de hardware tiene recursos limitados (CPU, RAM). Un VFD simple solo puede tener 5 "Instancias de Conexión" disponibles. Si un 6º PLC (o HMI) intenta hacer un Forward_Open, el Gestor de Conexión *rechazará* la solicitud con un error de "Recursos Insuficientes". Esto es un problema común de diagnóstico.
4. **Objeto Enrutador de Mensajes (Message Router - Clase 0x02)** 🗺️
   - Es el "servidor" o "dispatcher" subyacente. Recibe todas las peticiones de red y las enruta a la Clase/Instancia/Atributo correcto dentro del dispositivo.
   - Es crucial en dispositivos "puente" (bridges), como se mencionó en el Cap 1.0. Un

paquete CIP para un dispositivo en una sub-red se empaqueta con una "Ruta" (Path). Esta ruta es un "mapa" que el Enrutador de Mensajes sabe leer.
    - **Ejemplo de Ruta:** Port 1 -> Network 192.168.1.x -> Address 50 -> (backplane) -> Slot 3 -> (deviceNet) -> Node 10. El Enrutador de Mensajes en la IP .50 recibe el paquete, ve que no es para él, y lo "enruta" por su backplane al Slot 3, que a su vez lo re-empaqueta para DeviceNet.

Capítulo 1.2: Los Dos Tipos de Mensajería (La "Conversación")

CIP define dos formas fundamentalmente diferentes de comunicarse, optimizadas para tareas distintas.

1. Mensajería Explícita 💬

- **Propósito:** Tareas no urgentes, configuración, diagnóstico o lectura/escritura de un dato una sola vez. Es para datos que *no* son críticos en el tiempo.
- **Modelo:** Pregunta/Respuesta (Cliente/Servidor). Es como una llamada a un método (getter/setter) o una consulta a una base de datos. Se le llama "Unconnected Messaging" (UCMM) porque cada solicitud es una transacción completa e independiente.
- **Ejemplo:** El PLC envía un comando (usando el **Servicio** Get_Attribute_Single) al **Objeto Identidad** para leer el **Atributo** SerialNumber. El PLC *detiene* su lógica para esa tarea y *espera activamente* una respuesta. Si la respuesta no llega, genera un error. Los HMIs (pantallas) usan casi exclusivamente este método para rellenar los campos de una pantalla.
- **Transporte (en EtherNet/IP):** Usa **TCP/IP** (sobre el puerto **44818**). TCP garantiza la fiabilidad (los paquetes se confirman y se reordenan si llegan mal), pero esta sobrecarga de gestión (el "handshake") lo hace más lento e impredecible (no determinista).

2. Mensajería Implícita 📡

- **Propósito:** Control en tiempo real (E/S). Datos que deben se enviarse repetidamente a alta velocidad (ej. consigna de velocidad, posición de un eje).
- **Modelo:** Productor/Consumidor (Streaming). Es un modelo "publicar/suscribir". Se le llama "Connected Messaging" (Class 1) porque el "contrato" (Forward_Open) establece una "conexión" o sesión persistente.
- **Ejemplo:** El VFD (Productor) "publica" su "Velocidad Actual" (contenida en un **Objeto Ensamblado**) 50 veces por segundo (cada 20ms). El PLC (Consumidor) está "suscrito" a este flujo de datos y simplemente "escucha". No hay una "pregunta" por cada dato. Si un paquete se pierde, no importa, porque 20ms después llegará uno nuevo y actualizado.
- **Transporte (en EtherNet/IP):** Usa **UDP/IP** (sobre el puerto **2222**). UDP es rápido y "ligero" (sin handshake, sin reintentos). Simplemente "dispara y olvida". Esta velocidad lo hace ideal para el control en tiempo real, donde el *último* dato es el único que importa, no un dato antiguo que se retrasó.

Parte 2: La Práctica - Poniendo CIP a Trabajar

Capítulo 2.1: El "Contrato" - Cómo se Activa la Mensajería Implícita

Esta es la parte clave que une ambos tipos de mensajería: **se usa un mensaje Explícito 💬 para activar el flujo Implícito 📡.** Es una negociación.

1. **Negociación:** El PLC (Scanner) envía un mensaje **Explícito** 💬 especial llamado Forward_Open al **Objeto Gestor de Conexión (Clase 0x06)** del VFD (Adapter).
2. **El "Contrato":** Este mensaje es un bloque de datos complejo que define las reglas del "streaming":
   - **Los Ensamblados:** "Tú (VFD) usarás tu Ensamblado 150 para *producir* tu estado. Yo (PLC) usaré mi Ensamblado 100 para *producir* mis comandos."
   - **El RPI (Requested Packet Interval):** La velocidad del streaming (ej. "enviarnos datos cada 20 milisegundos"). Esto es una *solicitud*. El dispositivo receptor puede *aceptar* el RPI, o *rechazarlo* y proponer uno más lento si no es capaz de cumplirlo.
   - **El Timeout (Watchdog):** Qué hacer si se pierde la conexión. Se define un multiplicador (ej. 4). Si el RPI es 20ms, el timeout será 80ms. Si una de las partes no oye a la otra durante 80ms, la conexión se considera "cerrada" y el dispositivo pasa a un estado seguro (ej. el VFD detiene el motor).
   - **Los IDs de Conexión (Connection ID):** Si se acepta el contrato, se generan dos "Números de Serie de Conexión" (O->T y T->O Connection ID). Estos IDs son como "claves de sesión" (ej. 0x1A2B3C4D).
3. **Activación:** El VFD responde "Acepto" (otro mensaje Explícito), incluyendo su ID de Conexión. En ese instante, *ambos* dispositivos empiezan a *producir* (transmitir) sus respectivos ensamblados vía UDP 📡 al RPI acordado.
   - **Implicación de los IDs:** Cada paquete UDP 📡 que se envía *no* se identifica por su IP (eso es lento), sino que lleva este Connection ID en su cabecera. El dispositivo receptor usa este ID (un entero de 32 bits) para hacer una búsqueda instantánea en su tabla de conexiones y saber *inmediatamente* a qué sesión pertenece y en qué lugar de la memoria debe escribir los datos. Esto es fundamental para el alto rendimiento.
4. **Desactivación:** El flujo se detiene con un mensaje Explícito Forward_Close (una terminación "elegante", como colgar una llamada) o cuando se activa el *Timeout* (una terminación por error, como cuando se corta la llamada).

Capítulo 2.2: Los "Paquetes" - El Objeto Ensamblado (Assembly)

- **Definición:** Un Ensamblado es un "DTO" 📦, un contenedor que agrupa múltiples datos. Su formato (el orden de los bytes, el tipo de datos) debe ser idéntico y conocido por el Productor y el Consumidor.
- **Regla Clave:** Una conexión implícita tiene **un solo Ensamblado por dirección** (uno de Salida, O->T, uno de Entrada, T->O).
- **Flexibilidad:** Ese *único* Ensamblado puede estar compuesto por múltiples piezas de datos (ej. [Comando (2 bytes), Velocidad (2 bytes)] = paquete de 4 bytes). El formato exacto, incluyendo el tipo de dato (SINT, INT, DINT, REAL) y el orden (Endianness), es crítico.

- **Endianness:** CIP es oficialmente "Little-Endian" (el byte menos significativo va primero). Si un dispositivo de un mundo "Big-Endian" (como un sistema basado en Motorola) se conecta, los datos aparecerán intercambiados (0x1234 se leerá como 0x3412). Esto es una pesadilla de diagnóstico común.

Ensamblados Fijos vs. Dinámicos

- **Ensamblados Fijos (95% de los casos):** Son predefinidos por el fabricante en el *firmware* del dispositivo. Son como los "Combos" 🍔🍟 de un menú. Son rápidos, estándar, fiables y están optimizados en el firmware para un acceso directo a la memoria. El ingeniero simplemente selecciona uno de un menú desplegable en el software del PLC, que leyó las opciones del fichero EDS.
- **Ensamblados Dinámicos (Avanzado):** El PLC puede, programáticamente, "construir" un ensamblado "a la carta" 🥗 durante la negociación Forward_Open. El PLC envía una lista de los atributos específicos que desea. El Adapter debe tener la CPU y RAM suficientes para "montar" este paquete personalizado en tiempo real. Se usa a veces para aplicaciones de *data logging* (recogida de datos), pero casi nunca para control de máquina, ya que es complejo y rompe la estandarización.

Capítulo 2.3: La "Estandarización" - Perfiles y EDS

¿Cómo puede un PLC de "Marca A" saber qué Ensamblados "fijos" tiene un VFD de "Marca B"?

1. **Perfiles de Dispositivo (Device Profiles)** 📜
   - Este es el **"catalogo" oficial de ODVA**.
   - Es una "receta" 📝 que los fabricantes *deben* seguir para ser "compatibles". Por ejemplo, el "Perfil de Variador de CA" (AC Drive Profile) define los Objetos, Atributos y (lo más importante) los **Ensamblados Estándar** que todos los variadores compatibles deben tener.
   - **Ejemplo Concreto:** El Perfil de AC Drive (Clase 0x29) define que Atributo 8 es Motor_RPMs. Por lo tanto, un PLC puede ser programado para leer Clase 0x29, Instancia 1, Atributo 8 y *sabe* que obtendrá las RPM, sin importar si el VFD es Rockwell, Yaskawa o Schneider (siempre que cumpla el perfil).
   - También define la **Máquina de Estados** (el comportamiento) del dispositivo (ej. qué comandos causan una transición de "Idle" a "Running").
   - Esto es lo que garantiza la **intercambiabilidad** 🔄.
2. **Archivos EDS (Electronic Data Sheet)** 📄
   - Es el **"manual de instrucciones"** o "driver" de un dispositivo específico. Es un archivo de texto en formato .ini, legible por humanos.
   - El ingeniero carga este archivo en el software del PLC (ej. Studio 5000). El software *parsea* (analiza) el EDS y "aprende" todo sobre el dispositivo *antes* de conectarse a él.
   - El EDS contiene:
     - [Device]: La información del Objeto Identidad.

        - [Params]: La lista de todos los parámetros de configuración (Mensajería Explícita).
        - [Assembly]: La lista de todos los Ensamblados fijos (Mensajería Implícita) que ofrece.
    - **EDS vs. Dispositivo Real:** El dispositivo real (Adapter) *no* almacena toda la información del EDS (como las cadenas de texto "Velocidad Máxima" o "RPM"). Esto se hace para **ahorrar costes 💰 y memoria 💾** en el hardware (especialmente en sensores simples). El EDS vive en el PC y actúa como el "diccionario" o el "código fuente con comentarios" que da sentido a los datos en crudo (Parámetro[5]) que están en el dispositivo.
    - **Excepción (EDS Upload):** Algunos dispositivos de gama alta *sí* almacenan su propio EDS a bordo. El software de diagnóstico puede "subir" (upload) el EDS directamente desde el dispositivo, lo cual es útil si se ha perdido el archivo original.

Capítulo 2.4: Gestión de Conexiones Múltiples (Propiedad y Acceso)

Esta es una pregunta de arquitectura fundamental: ¿Pueden múltiples Scanners (PLCs, HMIs) acceder a un mismo Adapter (VFD)? La respuesta es **Sí, pero depende del tipo de acceso**.

El **Objeto Gestor de Conexión (Clase 0x06)** actúa como el "portero" 👮 que gestiona quién puede conectarse y qué puede hacer.

1. Acceso Explícito (TCP 💬 - Configuración / Diagnóstico)

- **Regla: Se permiten Múltiples Conexiones.**
- **Analogía:** Un servidor web. Un Adapter puede manejar numerosas conexiones TCP (Explícitas) al mismo tiempo, limitadas solo por su CPU y memoria.
- **Escenario Real:** Es perfectamente normal tener:
    - **PLC 1:** Leyendo un parámetro de fallo (Explícito) tras un error.
    - **HMI 1:** Leyendo el Atributo "Velocidad Actual" (Explícito) cada 500ms para un gráfico.
    - **PC de Ingeniería:** Conectado para leer y escribir parámetros de configuración (Explícito).
- Todos estos accesos coexisten sin conflicto.

2. Acceso Implícito (UDP 📡 - Control en Tiempo Real)

Aquí es donde se aplica la disciplina estricta para evitar el caos. Un Adapter no puede obedecer a dos amos. Las conexiones implícitas (Forward_Open) se dividen en tipos:

- **Tipo A: Propietario Exclusivo (Exclusive Owner) 🔑**
    - **Propósito:** Control total de la máquina. La conexión incluye tanto datos de Entrada (T->O) como de Salida (O->T).
    - **Regla: SOLO PUEDE HABER UN PROPIETARIO EXCLUSIVO.**
    - **Mecanismo:** El primer PLC (Scanner) que envía un Forward_Open pidiendo "Propiedad Exclusiva" la obtiene. El Gestor de Conexión le da la "llave" 🔑.

    - **Rechazo:** Si un segundo PLC intenta pedir "Propiedad Exclusiva", el Gestor de Conexión *rechazará* la solicitud con el error 0x110 ("Connection already owned/configured by another node").
    - **Consecuencia:** Esto es una característica de seguridad fundamental. Evita que dos PLCs envíen comandos contradictorios (uno "Arranca" y otro "Para") al mismo dispositivo.
- **Tipo B: Solo Escucha (Listen-Only)** 🎧
    - **Propósito:** Monitorización, no control. La conexión solo recibe los datos de Entrada (T->O) que el Adapter está produciendo.
    - **Regla: SE PERMITEN MÚLTIPLES CONEXIONES.**
    - **Mecanismo:** Un segundo (o tercero, o cuarto...) PLC o HMI envía un Forward_Open pidiendo una conexión "Listen-Only".
    - **Escenario Real:**
        - **PLC 1 (Controlador):** Tiene la conexión "Exclusive Owner". Envía comandos ([Arrancar, 1800 RPM]) y recibe el estado ([Corriendo, 1799 RPM]).
        - **PLC 2 (Monitor):** Pide una conexión "Listen-Only".
        - **Resultado:** El VFD ahora envía su paquete de estado ([Corriendo, 1799 RPM]) a *ambos* PLCs. El PLC 2 puede monitorizar la velocidad y el estado, pero *no puede* enviar comandos de control. Si el PLC 1 falla, el PLC 2 se da cuenta (por timeout) y puede iniciar una lógica de respaldo.
- **Tipo C: Solo Entrada (Input-Only)**
    - Similar a "Listen-Only", es una conexión de solo monitorización.
- **Tipo D: Solo Salida (Output-Only)**
    - Menos común. Un PLC (Productor) envía sus datos, pero no espera una respuesta (Consumidor). Útil para "latidos" (heartbeats) o enviar datos a un dispositivo simple que no responde.

Resumen de Arquitectura:
Un Adapter (VFD) puede tener simultáneamente:

- **1** Conexión de Propietario Exclusivo (el PLC de control).
- **Múltiples** Conexiones de Solo Escucha (PLCs de monitorización, HMIs de alta velocidad).
- **Múltiples** Conexiones Explícitas (HMIs de baja velocidad, PCs de ingeniería).

Parte 3: El Transporte - EtherNet/IP en el Mundo Real

Capítulo 3.1: La "Autopista" - Hardware de Red Industrial

No se puede usar un switch de oficina de 20€ para una red de control. El entorno físico (vibración, temperatura) y los requisitos de fiabilidad (determinismo) son diferentes. Se necesita hardware industrial.

1. Hardware Físico

- **Conectores:** Se prefieren conectores robustos como el **M12** (circular, con rosca, sellado IP67) sobre el **RJ45** (de plástico, frágil) en el campo. Un RJ45 fallará por vibración o

corrosión; un M12 está diseñado para resistirlo.
- **Cableado:** Se usa cableado de par trenzado apantallado (Shielded Twisted Pair - STP) de alta categoría (CAT5e, CAT6) para resistir la interferencia electromagnética (EMI) de los motores y variadores.

2. Switches Gestionados (Managed Switches) 🧠

Un switch "no gestionado" es una rotonda ⭕ (un "hub" tonto). Un switch "gestionado" tiene semáforos 🚦 y policía de tráfico. Proporciona herramientas vitales para el control industrial.

- **QoS (Quality of Service - Calidad de Servicio):** Resuelve la **CONGESTIÓN**.
  - Permite crear un "carril VIP" 💎. El dispositivo (PLC/VFD) *marca* sus paquetes de control 📡 con un valor DSCP (Differentiated Services Code Point) en la cabecera IP. El estándar es DSCP 46 (Expedited Forwarding, EF).
  - El switch gestionado se configura para *confiar* en esta marca. Cuando ve un paquete marcado como "46", lo pone en su "cola de prioridad" más alta, asegurando que salga *primero*, incluso si hay una tormenta de tráfico de video o diagnóstico.
  - Esto asegura que los comandos en tiempo real nunca se retrasen (tengan "jitter") por congestión.
- **VLANs (Redes Virtuales):** Permite segmentar la red. Se puede crear una VLAN para "Control" (PLCs y VFDs) y otra para "Información" (HMIs, conexión a la planta). Esto mejora la seguridad (las redes no se ven entre sí) y el rendimiento (reduce el tráfico irrelevante).
- **Port Mirroring:** Permite al switch copiar todo el tráfico de un puerto (ej. el del PLC) a otro puerto (ej. donde conectas tu portátil). Es la función que habilita el diagnóstico con Wireshark.
- **Topologías de Red:** Los puertos dobles en los dispositivos (que actúan como un mini-switch interno) permiten:
  - **Topología Lineal (Daisy-Chain) ⛓️:** Ahorra costes de cableado, pero no es redundante. Un fallo en un dispositivo desconecta a todos los siguientes.
  - **Topología de Anillo (Ring) 💍:** Proporciona redundancia.

3. Métodos de Envío: Unicast, Broadcast y Multicast

Esta es una función de red crítica que los switches gestionados deben manejar correctamente.

- **Unicast (Uno-a-Uno) 📞**
  - **Cómo funciona:** Una "llamada telefónica". El paquete va desde una IP de origen (ej. PLC) a *una* IP de destino (ej. VFD).
  - **Uso en CIP:** Es el método más común. Toda la **Mensajería Explícita (TCP)** es Unicast. La *mayoría* de la **Mensajería Implícita (UDP)** también es Unicast (una conexión Punto-a-Punto entre el PLC y el VFD).
- **Broadcast (Uno-a-Todos) 📣**
  - **Cómo funciona:** "Gritar en una habitación". El paquete va desde una IP de origen a *todas* las IPs de la subred (dirección 255.255.255.255).

    - **Problema:** Es muy "ruidoso" e ineficiente. *Cada* dispositivo en la red (cada PLC, VFD, HMI, PC) tiene que recibir y procesar ese paquete en su CPU, aunque no le interese.
    - **Uso en CIP:** Se usa *muy* poco, solo para **descubrimiento de dispositivos**. Cuando un software como RSLinx arranca, puede enviar un Broadcast (un ListIdentity) que "grita": "¡Hola! ¿Hay algún dispositivo CIP en esta red?".
- **Multicast (Uno-a-Muchos)** 📻
    - **Cómo funciona:** La solución "inteligente" para el modelo Productor/Consumidor 1-a-Muchos. Es como un "walkie-talkie en un canal específico".
    - **El Escenario:** Imagina que tienes un **Productor** (ej. un sensor láser 📏) y **5 Consumidores** (5 PLCs) que necesitan leer ese dato al mismo tiempo.
    - **La Solución (Multicast):** El sensor (Productor) envía **un solo paquete** 📡 a una dirección IP de Multicast (ej. 239.192.1.10), que es como un "canal de radio". Los 5 PLCs (Consumidores) se "suscriben" a ese canal.
    - **El Problema (Switches "Tontos"):** Un switch no gestionado no sabe qué puertos están suscritos. Trata el Multicast como si fuera un Broadcast y lo **inunda** 🌊 por *todos* sus puertos, creando una "tormenta de tráfico" que colapsa la red.
    - **La Solución (Switch Gestionado): IGMP Snooping**. "IGMP" es el protocolo de suscripción, y "Snooping" (espiar) 🕵️ significa que el switch "espía" esas suscripciones. El switch construye una tabla y sabe que solo los puertos 3, 5 y 7 están escuchando el canal 239.192.1.10. Cuando recibe un paquete para ese canal, *solo* lo reenvía a esos tres puertos, protegiendo al resto de la red.

4. DLR (Device Level Ring)

- **DLR (Anillo a Nivel de Dispositivo):** Resuelve **FALLOS FÍSICOS**.
- Los dispositivos (o un switch gestionado que actúa como "Supervisor") se conectan en un bucle 💍. El Supervisor envía "beacons" (paquetes de salud) para verificar la integridad del anillo y bloquea el tráfico en un punto para evitar bucles.
- Si un cable se rompe 💥, los beacons dejan de llegar. El Supervisor lo detecta (en < 3ms) y desbloquea su puerto, permitiendo que el tráfico fluya por el otro lado del anillo.
- **vs. RSTP (Protocolo IT):** Un protocolo de IT como RSTP (Rapid Spanning Tree Protocol) puede tardar *segundos* en re-converger (encontrar una nueva ruta). En una máquina, 2 segundos de ceguera es una eternidad; todas las conexiones Implícitas habrían caído por *Timeout*. DLR está diseñado para una convergencia de < 3ms, que es más rápida que el RPI más rápido, por lo que la red se "cura" sin que las conexiones se caigan.
- **QoS vs. DLR:** Se necesitan ambos. DLR es la "ruta de desvío" 🚗 y QoS es el "carril VIP" 💎. DLR asegura que la autopista esté abierta; QoS asegura que el tráfico importante vaya rápido en esa autopista.

Capítulo 3.2: El "Diagnóstico" - Viendo el Tráfico

- **Wireshark 🕵️:** Es la herramienta principal para analizar el tráfico de red. Con un "port mirror" en un switch gestionado, un técnico puede ver todo.
- **Filtros Prácticos en Wireshark:**
    - **Mensajería Explícita:** tcp.port == 44818. Se verán peticiones [SYN], [SYN, ACK] y

luego paquetes [PSH, ACK] con datos enip (CIP) que se pueden decodificar para leer Get_Attribute_Single.
    - **Mensajería Implícita:** udp.port == 2222. Se verá un "río" de paquetes. La columna "Time" es clave: la diferencia entre paquetes debe ser *exactamente* el RPI (ej. 0.020s, 0.020s, 0.020s). Si se ve 0.040s, se ha perdido un paquete. Si se ve 0.021s, 0.019s, hay "jitter" en la red.
- **Otros Herramientas:** El software del PLC (ej. Studio 5000) y el software de diagnóstico (ej. RSLinx) tienen herramientas gráficas para ver el estado de la red, los contadores de paquetes y los fallos de conexión. Un técnico puede ver si los contadores de "paquetes perdidos" están incrementando.

Parte 4: Las Capas Superiores (Funciones Avanzadas de CIP)

CIP es "Común" porque también sirve de base para funciones avanzadas, permitiendo que todas operen sobre el mismo cable.

Capítulo 4.1: CIP Safety ⛑️ (Seguridad Funcional)

- **Propósito:** Proteger a las personas **DE** la máquina (fallos, accidentes). Se rige por estándares como IEC 61508 (SIL).
- **Problema:** ¿Cómo enviar un Paro de Emergencia 🚨 por una red UDP no fiable? Un bit corrupto podría cambiar un "Parar" por un "Arrancar".
- **Solución:** Se usa un "Canal Negro" (Black Channel). La red (el "canal negro") se asume como 100% no fiable. La *inteligencia* está en los dispositivos finales (Safety PLC, Safety VFD).
    - Los datos de seguridad se envían con alta redundancia: datos y sus inversos ([Parar=1, NoParar=0]), marcas de tiempo (para evitar paquetes retrasados), números de secuencia (para evitar paquetes perdidos o re-inyectados) y un CRC de alta integridad.
- **El Controlador:** Un PLC de seguridad especial (rojo 🟥, ej. GuardLogix) comprueba todo. Si un paquete se retrasa, se corrompe (los datos y su inverso no coinciden) o se pierde (el número de secuencia se salta), el PLC activa el estado seguro (paro).
- **Requiere:** Una cadena de seguridad completa (botón, PLC y variador certificados para CIP Safety y un nivel SIL, ej. SIL 3).

Capítulo 4.2: CIP Sync ⏱️ (Sincronización)

- **Propósito:** Proteger contra el **tiempo** (jitter o retrasos variables).
- **Problema:** ¿Cómo hacer que 20 robots 🤖 muevan un cristal *exactamente* al mismo microsegundo?
- **Solución:** Usa el estándar **IEEE-1588 (PTP - Precision Time Protocol)**. Un "Reloj Maestro" (Grandmaster Clock) 🕰️ sincroniza los relojes internos de todos los dispositivos. Un algoritmo (Best Master Clock Algorithm) elige al maestro

automáticamente.
    - **La Magia (vs. NTP):** Un protocolo simple como NTP solo sincroniza la hora. PTP (IEEE-1588) es mucho más. Mide activamente el *retraso del cableado* (latencia) entre el maestro y cada esclavo (con mensajes Delay_Req y Delay_Resp) y *compensa* por él. Esto le permite alcanzar una precisión de **sub-microsegundo**.
- **La Aplicación (CIP Motion):** Esta es la *aplicación* que se construye sobre CIP Sync. El PLC envía comandos "programados" (ej. "Mover a las 12:30:05.123456"), y todos los dispositivos los ejecutan en ese instante exacto. Permite un control de movimiento multi-eje coordinado y distribuido a través de la red, algo que antes requería cableado analógico dedicado.

Capítulo 4.3: CIP Security 🔒 (Ciberseguridad)

- **Propósito:** Proteger a la máquina **DE** las personas (hackers, atacantes, malware).
- **Problema:** CIP es un protocolo "confiado". Cualquiera en la red puede enviar un comando Forward_Open o un paquete UDP malicioso.
- **Solución:** Adopta la estrategia de "Defensa en Profundidad" (DiD), añadiendo las capas de seguridad de internet:
    - **Autenticación 🪪:** Los dispositivos usan certificados digitales (estándar X.509) para probar quiénes son. Un portátil desconocido no puede conectarse.
    - **Integridad 📜:** Los mensajes se "firman" (usando HMAC) para que no puedan ser modificados en tránsito.
    - **Confidencialidad 🤫:** El tráfico se puede cifrar (encriptar, usando TLS/DTLS) para que no pueda ser espiado.
- **El "Trade-off":** El cifrado añade **latencia** (retraso). Un VFD tiene una CPU optimizada para el control de motores, no para criptografía. Pedirle que cifre/descifre 100 paquetes UDP por segundo consume una CPU valiosa y añade "jitter", lo que puede *romper* el control en tiempo real (aumenta el RPI mínimo).
- **Safety vs. Security:** Son opuestos.
    - **Safety ⛑️** quiere *detener* la máquina ante un error (ej. un paquete retrasado).
    - **Security 🔒** *crea* retrasos (debido al cifrado), lo que puede *causar* un disparo de Safety.
    - Por esto, la ciberseguridad en la red de control (OT) se suele gestionar en el "perímetro" (Firewalls, VLANs) en lugar de cifrar cada paquete.

Parte 5: El Ecosistema (La Competencia)

- **Modbus TCP:** El abuelo de los protocolos Ethernet. Es extremadamente simple (solo lee/escribe "registros"). No tiene modelo de objetos, ni perfiles, ni mensajería implícita. Sigue siendo muy usado para dispositivos simples (medidores de energía) donde la velocidad no es crítica. Su simplicidad es su fortaleza (cualquiera puede implementarlo) y su debilidad (es un modelo "poll-response", no "publish-subscribe").
- **PROFINET 🚀 (de Siemens):** El gran competidor.
    - **PROFINET RT (Real-Time, Class B):** Se "salta" el TCP/IP (Capas 3 y 4) y envía datos

directamente en la trama de Ethernet (Capa 2) usando un EtherType especial (0x8892). Es rápido. Es el equivalente funcional de EtherNet/IP estándar.
    - **PROFINET IRT (Isochronous Real-Time, Class C):** Es el equivalente a CIP Motion. Usa switches con hardware especial para crear "ranuras de tiempo" (time slots) fijas, reservando ancho de banda. Esto garantiza un determinismo absoluto (cero jitter).
- **EtherCAT 🏎️ (de Beckhoff):** El demonio de la velocidad.
    - **Cómo funciona:** Usa un método de "procesamiento al vuelo" 🚆 donde un *único* paquete Ethernet (el "tren") sale del Maestro.
    - El tren atraviesa al Esclavo 1, que tiene un ASIC (hardware) especial. El ASIC "saca" sus datos del tren y "pone" sus nuevos datos de vuelta, *todo en nanosegundos* mientras el paquete pasa.
    - El tren atraviesa al Esclavo 2, 3... hasta el final de la línea.
    - El último esclavo "da la vuelta" al paquete y lo envía de regreso.
    - Cuando el "tren" regresa al Maestro, contiene la imagen de E/S *completa* y actualizada de toda la red.
    - **Implicación:** Es extremadamente rápido, eficiente y determinista, pero requiere hardware ASIC especial en cada dispositivo.

Parte 6: Implementación Práctica y Diagnóstico (La Perspectiva del Scanner)

Habiendo cubierto la teoría del protocolo y la infraestructura de red, esta sección se centra en la perspectiva del ingeniero y programador que utiliza un "Scanner" (PLC/PAC) para construir y mantener un sistema.

Capítulo 6.1: Programación en el PLC (El "Cerebro")

¿Cómo interactúa un programador con estos conceptos en un entorno como Rockwell Studio 5000?

1. Mapeo de Memoria (Tag Mapping)

- **De `byte[]` a `Tag` Estructurado:** Cuando añades un Adapter (VFD) a tu árbol de E/S usando su fichero EDS 📄 y seleccionas un Ensamblado 📦 (ej. "Basic Speed Control"), el software del PLC hace algo mágico: **auto-genera** dos etiquetas (tags) estructuradas en la base de datos del controlador.
    - `Mi_VFD:I` (para las Entradas / T->O)
    - `Mi_VFD:O` (para las Salidas / O->T)
- **La "Imagen de Datos":** El sistema operativo del PLC (el firmware) maneja la red de forma asíncrona a la lógica. *Antes* de que comience el "scan" de tu programa de lógica, el PLC:
    1. Recibe el último paquete UDP 📡 del VFD.
    2. Verifica el `Connection ID`.
    3. Realiza un `memcpy` de los bytes del paquete UDP directamente en la estructura de memoria de la etiqueta `Mi_VFD:I`.

- **Consecuencia:** Cuando tu lógica lee Mi_VFD:I.VelocidadActual, no está accediendo a la red; está leyendo un valor en su propia memoria (RAM) que fue actualizado justo antes de que comenzara el scan. Esto es extremadamente rápido y determinista.

2. La Relación Crítica: RPI vs. Scan Time

Este es un concepto fundamental del rendimiento.

- **RPI (Requested Packet Interval):** Es la velocidad de la red (UDP). Define qué tan "frescos" son los datos en la imagen de memoria. Se configura por dispositivo.
- **Tiempo de Scan (Scan Time):** Es la velocidad de tu programa de lógica (Rung 1 -> Rung N). Define qué tan *rápido* reacciona tu lógica a esos datos. Se configura por Tarea (Task).
- **Escenarios:**
  - **RPI (10ms) < Scan (20ms):** La red es más rápida que la lógica. El PLC puede recibir *dos* paquetes UDP (@ 10ms, @ 20ms), pero la lógica solo se ejecutará *una vez* (@ 20ms) y solo verá el segundo paquete. Es un desperdicio de ancho de banda.
  - **RPI (20ms) > Scan (5ms):** La lógica es más rápida que la red. Tu programa de lógica se ejecutará *cuatro veces* (@ 5ms, 10ms, 15ms, 20ms) utilizando datos *idénticos y obsoletos* antes de que el RPI (@ 20ms) actualice la imagen de datos con un nuevo valor.
  - **Práctica Recomendada:** El RPI de la red debe ser igual o ligeramente más rápido que el tiempo de scan de la tarea que lo consume.

3. La Instrucción MSG (Mensajería Explícita 💬)

- **Cómo se usa:** Para enviar un Mensaje Explícito 💬 (ej. leer un parámetro que *no* está en el Ensamblado 📦), se usa una instrucción de lógica llamada MSG.
- **Configuración:** No es automática; el programador la configura manualmente. Se crea una etiqueta de control (tipo MESSAGE) donde se especifica:
  - **Servicio:** Get_Attribute_Single (Código 0x0E)
  - **Clase:** 0x01 (Identity) o 0x29 (AC Drive)
  - **Instancia:** 1
  - **Atributo:** 6 (SerialNumber) o 8 (Motor_RPMs)
  - **Destino:** La etiqueta del PLC donde se almacenará la respuesta.
  - **Ruta (Path):** La dirección IP del Adapter.
- **Ejecución:** La instrucción MSG es asíncrona. Cuando el programa la activa (.EN), el copilador de comunicaciones del PLC envía el paquete TCP 💬 y la lógica sigue ejecutándose. El programador debe monitorear los bits de la etiqueta de control (.DN - Hecho, o .ER - Error) para saber cuándo ha llegado la respuesta.

Capítulo 6.2: Arquitectura de Red (Convergencia IT/OT)

La red de control (OT - Operational Technology) ya no está aislada. Debe conectarse de forma segura a la red de empresa (IT - Information Technology) para el análisis de datos.

- **El Modelo Purdue (Simplificado):**
  - **Nivel 4 (IT):** Servidores de la empresa, ERP.

    - **Nivel 3.5 (DMZ):** Una "zona desmilitarizada" segura entre IT y OT.
    - **Nivel 2 (Control):** La red de PLC, VFDs, HMIs (nuestra red EtherNet/IP).
    - **Nivel 1 (Dispositivos):** Los sensores/actuadores físicos.
- **Firewalls y DMZ (Defensa en Profundidad):** La mejor práctica.
    - Un **Firewall IT/DMZ** se sitúa entre la red de empresa (L4) y la DMZ (L3.5).
    - Un **Firewall OT/DMZ** (un dispositivo industrial) se sitúa entre la DMZ y la red de control (L2).
    - **Reglas:** El firewall OT se configura para *bloquear todo* por defecto. Luego, se abre explícitamente solo lo necesario.
        - **Ejemplo de Regla:** PERMITIR IP_Servidor_SQL (en DMZ) iniciar TCP puerto 44818 (Explícito 💬) a IP_PLC_Línea_1 (en OT), pero DENEGAR IP_Servidor_SQL iniciar UDP puerto 2222 (Implícito 📡). Esto permite a IT "preguntar" por datos, pero le *prohíbe* enviar comandos de control.
- **NAT (Network Address Translation):**
    - **El Problema:** Un fabricante de máquinas (OEM) quiere entregar su máquina con una red pre-configurada y probada (ej. 192.168.1.x). Pero la fábrica del cliente ya usa esa red.
    - **La Solución:** El OEM incluye un dispositivo NAT (un switch gestionado o un router) en su máquina. La máquina funciona en su red 192.168.1.x. El cliente solo conecta un cable de su red (10.20.30.x) al puerto "WAN" del dispositivo NAT. El dispositivo NAT "traduce" las direcciones, permitiendo que la máquina se integre sin necesidad de re-direccionar todos los VFDs.

Capítulo 6.3: Diagnóstico Práctico y Errores Comunes

En la práctica, la teoría falla. Así es como se diagnostica.

1. La Herramienta #1: El LED "NS" (Network Status)

Todo dispositivo Adapter tiene un LED de estado de red (NS). Es la primera herramienta de diagnóstico.

- **Apagado:** Sin energía, o sin dirección IP.
- **Verde Fijo 🟢:** ¡El estado deseado! El dispositivo tiene IP y **una o más conexiones implícitas 📡 (Propietario/Listen-Only) están activas** y funcionando.
- **Verde Parpadeante ❇️:** El dispositivo tiene IP, pero **NO hay conexiones implícitas**. Solo tiene conexiones explícitas 💬.
    - *Diagnóstico:* Este es el estado normal de un PC o un HMI que solo está "preguntando" datos. Si un VFD está en este estado, significa que el PLC *no* ha logrado establecer la conexión Forward_Open.
- **Rojo Fijo 🔴:** Fallo crítico. Causa más común: **Conflicto de IP**. Dos dispositivos tienen la misma IP.
- **Rojo Parpadeante 🔻:** El error más común. La conexión implícita 📡 **ha fallado por Timeout (Watchdog)** ⏳.
    - *Diagnóstico:* La conexión Forward_Open *tuvo* éxito (el LED estuvo verde), pero luego

el "latido" UDP se detuvo. Esto apunta a:

1. Un cable de red roto o desconectado.
2. Un fallo en la topología DLR (el anillo no se "curó").
3. Congestión de red extrema (los paquetes UDP prioritarios (QoS) se están perdiendo).
4. El PLC (Scanner) se puso en modo "Program" o se apagó.

2. Códigos de Error Comunes del PLC (en Studio 5000)

Cuando un dispositivo muestra un triángulo amarillo ⚠️ en el árbol de E/S, el PLC reporta un código de error.

- **Code (16#0204) Connection timed out**
  - **Qué significa:** El "LED Rojo Parpadeante" 🔻. La conexión estaba activa y se perdió el latido UDP 📡.
  - **Causas:** Ver arriba (cable, congestión, PLC en modo Program).
- **Code (16#0110) Connection already owned by another node**
  - **Qué significa:** "Ya tengo un Propietario Exclusivo 🔑".
  - **Causa:** Como se vio en el Cap 2.4, un segundo PLC (o un HMI configurado erróneamente) está intentando tomar control "Exclusive Owner" de un VFD que ya está controlado por tu PLC principal.
- **Code (16#0109) Invalid Connection Size**
  - **Qué significa:** Los "Ensamblados" (DTOs 📦) no coinciden.
  - **Causa:** El PLC (basado en el EDS) le dijo al VFD: "Vamos a intercambiar un paquete de 32 bytes". El VFD (basado en su firmware) respondió: "Lo siento, ese Ensamblado solo tiene 28 bytes".
  - **Solución:** Tienes una discrepancia entre el fichero EDS y el firmware real del VFD. (Opciones de ensamblado incorrectas, o necesitas actualizar el EDS/firmware).
- **Code (16#0203) Connection request timed out**
  - **Qué significa:** El PLC envió el Forward_Open 💬 (la solicitud de contrato) pero el VFD *nunca* respondió.
  - **Causas:**
    1. Dirección IP incorrecta.
    2. Cable de red desconectado.
    3. El VFD está apagado.